

SmartRoom

an IoT solution for energy saving

FEDERICO CALZOLARI &
MATTIA SEGRETO

1 Introduction

In recent years, the growing focus on energy efficiency and environmental sustainability has played a key role in the design of intelligent indoor environments, both in residential and commercial contexts. Buildings are among the largest consumers of energy worldwide: according to estimates from the International Energy Agency (IEA), the building sector accounts for approximately 30–40% of total global energy consumption and a significant share of CO₂ emissions. In this context, developing technological solutions that enable more precise control over energy usage and resource management is becoming increasingly important.

The adoption of Internet of Things (IoT) technologies in smart home and building applications represents a concrete step toward this goal. IoT enables the creation of responsive and adaptive environments capable of monitoring environmental parameters and human activity in real time, and reacting accordingly to optimize comfort while minimizing energy waste. Lightweight communication protocols such as the *Constrained Application Protocol (CoAP)* are particularly suited for this purpose, as they allow efficient and low-power communication between constrained devices typically used in smart environments.

The project presented in this work fits into this technological and research framework, proposing an integrated solution for energy-efficient management within a smart room. The system consists of two main modules:

- **Environmental Module:** Equipped with temperature and humidity sensors, this module collects real-time environmental data to estimate the *perceived temperature*, a more accurate indicator of thermal comfort than raw temperature alone. Based on this estimation, the system can automatically activate heating or air conditioning systems to maintain optimal comfort while avoiding unnecessary energy use.
- **Lighting Module:** This module includes a manual light switch and a presence detection sensor. Once the light is turned on, the system starts monitoring the room for human activity. If no movement is detected after a set period of time, the lights are automatically turned off to save energy.

These modules communicate via a CoAP-based network, enabling distributed, efficient, and scalable control that adapts dynamically to changing environmental conditions and user behavior. The overarching goal of the project is to demonstrate that even with low-cost, resource-constrained devices, it is possible to implement intelligent systems that promote *energy conservation* without compromising user comfort.

2 Architecture overview

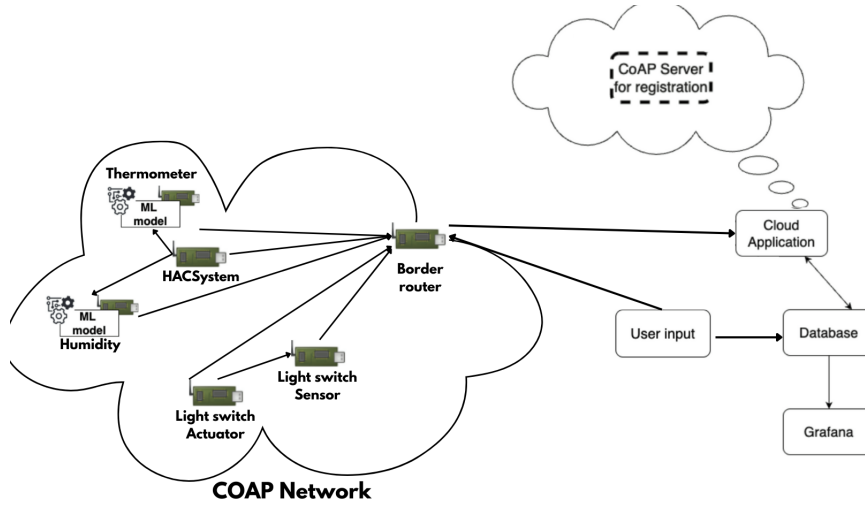


Figure 1: High-level architecture of the smart room system.

2.1 CoAP Network

2.1.1 Application Protocol Choice

The first step in designing the sensor network was selecting the appropriate *application-layer protocol*, which plays a crucial role in ensuring the system's efficiency, simplicity, and responsiveness. The chosen solution is the *Constrained Application Protocol (CoAP)*, a lightweight protocol specifically designed for environments with resource-constrained devices.

The main reasons for selecting CoAP are as follows:

- **RESTful model:** CoAP follows a web-like, resource-oriented approach where each system functionality is represented as a URI-accessible resource. This makes the design modular and intuitive. For instance, temperature data can be exposed via `/temperature`, while lights can be controlled through `/light` endpoints.
- **Efficiency and low overhead:** CoAP operates over UDP and uses a compact message format, minimizing network load and energy consumption. At the same time, it ensures reliability through mechanisms such as Confirmable (CON) messages and acknowledgments.
- **Automatic updates (Observe):** CoAP supports a built-in mechanism that allows a client to observe a resource and receive notifications whenever its state changes. This is particularly useful in reactive scenarios, such as automatically turning off lights after a period of inactivity, without the need for continuous polling.
- **Direct node-to-node communication:** Unlike other messaging protocols, CoAP does not require a central server or broker. Modules can communicate directly within the local network, simplifying the architecture and reducing potential points of failure.
- **No need for external connectivity after discovery:** Once the initial resource discovery phase is complete, CoAP nodes can continue to communicate directly within the local network without requiring external access beyond the border router. This allows the system to operate in a fully local and self-contained manner, ideal for our scenario. As a result, even if the cloud application becomes unavailable, the core functionalities of the system remain operational, ensuring robustness and continued performance in isolated or offline conditions.

These features make CoAP particularly well-suited for the distributed system proposed in this project, where efficient communication, low power usage, and ease of integration are key design requirements.

Why not MQTT? Although MQTT is a widely used protocol in IoT applications, it was not selected for this project due to several architectural and

functional considerations. MQTT is based on a publish/subscribe model that requires a central message broker to manage communication between nodes. This adds complexity and introduces a single point of failure, which is not ideal in small, self-contained systems like the one presented here. In contrast, CoAP provides direct, peer-to-peer communication enabling efficient and reactive interactions without relying on an external infrastructure.

Additionally, MQTT relies on a centralized broker that must remain reachable by all devices, often requiring communication that extends beyond the local network. This dependency introduces a need for continuous external access through the border router, making the system less autonomous and with a single point of failure.

For these reasons, CoAP was considered more suitable for a simple, modular, and energy-efficient architecture based on a limited number of devices operating within a local network.

2.1.2 Sensor Configuration

The system is structured into two main modules, each incorporating a set of sensors that support environmental monitoring and energy-aware automation. These sensors not only collect real-time data but also expose their state and control interfaces as CoAP resources. This enables distributed control logic and seamless interaction between nodes.

- **Environmental Module**

- **Thermometer**

- Measures ambient temperature. In addition, the node integrates a lightweight machine learning model based on autoregressive time series forecasting, enabling short-term prediction of temperature trends. This predictive approach allows the system to anticipate thermal changes and preemptively helps to activate heating or cooling actions.

ResourceName	Obs.	Method(s)	Call Format	Description
last value	Yes	GET	GET /temp	Returns current ambient temperature
temperature forecast	Yes	GET	GET /predh	Returns predicted temperature using AR model
on	No	GET/POST	GET/POST /ont	Turns on the temperature sensor or check if is on
off	No	GET/POST	GET/POST /offt	Turns off the temperature sensor or check if is on

– **Humidity sensor**

Monitors the relative humidity of the environment. As with the temperature sensor, an autoregressive model is applied locally to predict future humidity values. These predictions are used to compute a more accurate estimate of perceived temperature.

ResourceName	Obs.	Method(s)	Call Format	Description
last value	Yes	GET	GET /hum	Returns current relative humidity
humidity forecast	Yes	GET	GET /predh	Returns predicted humidity using AR model
on	No	GET/POST	GET/POST /onh	Turns on the humidity sensor or check if is on
off	No	GET/POST	GET/POST /offh	Turns off the humidity sensor or check if is on

– **HAC System (Heating and Cooling System)**

A responsive component that adjusts the room temperature based on the predicted comfort level. It compute perceived temperature using the temperature eand humidity forecasts.

ResourceName	Obs.	Method(s)	Call Format	Description
status	Yes	GET	GET /sts	Returns current HAC system state
set new thresholds	No	POST	POST /set_lim	Sets comfort temperature limits
get actual threshold	No	GET	GET /get_lim	Gets configured comfort temperature limits
on	No	GET/POST	GET/POST /onhac	Turns HAC system on or check if is on
off	No	GET/POST	GET/POST /offhac	Turns HAC system off or check if is on

• **Lighting Module**

– **Light sensor**

Activated manually via a physical button, this sensor initiates the lighting cycle and enables the presence detection logic. It serves as the link between user interaction and automated sensing behavior. Once the light is turned on, the presence detection logic aims to detect human movement within the room. This sensor monitors for activity and enables automatic light deactivation after a specified period of inactivity.

ResourceName	Obs.	Method(s)	Call Format	Description
light status	YES	GET	POST /light	Returns status changes
on	No	GET/POST	POST onlightsens	Activates the light sensor system or check if is on
off	No	GET/POST	POST /offlightsens	Deactivates the light sensor system or check if is on

– **Light actuator**

While not a sensor, the actuator is responsible for switching the light on or off based on signals from the light and presence sensors.

Name	Obs.	Method(s)	Call Format	Description
led status	No	GET/POST	GET/POST /led	Gets the current light state
on	No	GET/POST	GET/POST /onlightact	Turns the light actuator on or check if is on
off	No	GET/POST	GET/POST /offlightact	Turns the light actuator off or check if is on

2.2 Data Encoding Strategy

The system adopts two distinct encoding formats for different communication purposes between sensor nodes and the cloud application.

Registration Payloads

During the registration phase, each node sends a CoAP `POST` request to the `/registration` endpoint, including a JSON payload that encapsulates the node's metadata (SensorName and SensorServiceName). A typical registration message has the following structure:

```
{
  "s": "thermometer",
  "ss": ["temp", "predt", "offt", "ont"]
}
```

Sensor Data Payloads

For data transmission, each resource sends updates using a lightweight and uniform format: the payload is always transmitted as a JSON object with a single field:

```
{ "value": "string" }
```

This design ensures consistency across all resource types and simplifies processing within the cloud application, which can uniformly extract and interpret the payloads.

- **Numeric resources:** values such as temperature, humidity, or predictions are encapsulated as strings within the JSON structure (e.g., `{"value": "24.75"}`), and subsequently parsed into `float` format for storage and processing.

- **Categorical resources:** values representing discrete states or operational modes (e.g., "ON", "COOLING") are similarly sent as strings (e.g., {"value": "ON"}) and stored without transformation.

This uniform JSON-based approach simplifies client and server logic, enabling generic handling of data across heterogeneous sensor types.

Why Not XML? Firstly, XML is significantly more verbose than JSON. Even for simple data structures, it requires opening and closing tags that introduce unnecessary overhead. This verbosity becomes a serious issue in contexts where payload size is critical. Protocols like CoAP, widely used in IoT applications, impose strict size constraints (e.g., 64 bytes per packet), and operating systems like Contiki, designed for embedded devices with extremely limited resources, benefit greatly from compact data representations. JSON, with its concise syntax, helps minimize transmitted data and is better suited for environments with tight memory and bandwidth constraints.

Secondly, XML parsing is more complex and resource-intensive, requiring more memory and CPU cycles. In contrast, JSON is easier to decode and better supported across modern programming languages, making integration simpler on both edge devices and cloud platforms.

2.3 Cloud Application

The cloud application was developed in Java using the Californium library, which provides an implementation of COAP. Its primary role is to coordinate the sensor infrastructure by handling three core functionalities:

1. **Registration Service (/registration):** sensor nodes send a COAP POST request to this endpoint to register themselves. During the registration, metadata such as the node's IP address, type, and available resources are stored in a SQL database.
2. **Service Discovery (/service):** client applications can perform COAP GET requests on this endpoint to retrieve the IP addresses of registered nodes.
3. **Real-Time Observation:** for each registered sensor resource, the cloud application establishes a COAP observation thread. These observer threads subscribe to the nodes' observable resources (e.g., temperature, humidity, prediction) and listen for updates in real time.

2.4 User Application

At startup, the application initiates a cyclical discovery mechanism, which periodically queries the `ipv6_addresses` table in an SQL database to verify the availability of all services listed in the `REQUIRED_SERVICES` constant. These services represent key components necessary for the correct operation of the

system. As long as one or more of these services are missing, the application remains in an active waiting (polling) state, notifying the user that the system is not yet ready. Once all required services are registered and reachable, the discovery cycle is terminated, and the user is presented with the main application menu.

The **interactive menu** provides a simple command-line interface through which users can monitor the state of CoAP devices on the network and issue real-time control commands. The menu includes the following options:

1. **Display active services**

This option displays the current list of IPv6 services registered in the database. Each entry corresponds to an available service in the network, including its symbolic identifier and associated IP address. This feature is particularly useful for diagnostics and verification of node availability.

2. **Sensor and actuator ON/OFF control**

The user can activate or deactivate devices: light sensor, temperature sensor, humidity sensor, lighting actuator, and the HAC system. Beyond basic toggling, the application also manages **logical dependencies between devices**. For instance, when turning on a temperature sensor (**ont**) or humidity sensor (**onh**), if the HAC system is already active (**onhac**), it is automatically restarted (turned off and back on) to ensure consistent initialization and coordinated operation. This mechanism prevents inconsistent device states and preserves the overall reliability of the distributed system.

3. **View and modify HAC threshold settings**

This option allows users to query the current minimum and maximum threshold values for the HAC system via the **get_lim** command. Users can also update these values by entering new limits, which are formatted and validated before being sent to the system using a CoAP **POST** request via the **set_lim** command.

4. **Monitor HAC system status**

The system's current operational status can be queried using the **sts** command, providing real-time feedback on the HVAC module.

5. **LED control and monitoring**

Users can check the current LED state (on/off) and modify it by entering a binary value (0 for OFF, 1 for ON). This functionality serves as a simple CoAP actuator example and is useful for basic tests and debugging.

All communication with devices is performed using **CoAP GET and POST requests**, leveraging the **Eclipse Californium** library, which provides a straightforward yet powerful interface for interacting with IoT nodes. The IPv6 addresses are dynamically resolved at runtime using data from the database, allowing the application to operate flexibly and scale efficiently in dynamic network environments.

2.5 Database Structure

The MySQL database consists of a centralized registry table for node-resource associations and individual log tables for each resource.

Resource Registration Table

Table 1: Schema of the `ipv6_addresses` table

Column	Type	Description
<code>nodeip</code>	String	IPv6 address of the node
<code>nodename</code>	String	Logical identifier for the node
<code>resource</code>	String	URI of the exposed resource (e.g., <code>/temp</code>)

Per-Resource Log Tables

For each resource registered, a dedicated log table is created with the name pattern `{resource}_log`. This table stores all observed values over time.

Table 2: Schema of the `{resource}_log` table

Column	Type	Description
<code>value</code>	FLOAT or String	Measured value (numeric or string)
<code>timestamp</code>	TIMESTAMP	Time of insertion

This schema allows for scalable time series storage and easier visualization on Grafana, where each sensor resource is logged independently, while maintaining a centralized lookup registry for discovery and coordination.

3 Machine Learning Models: Predictive Modeling Approach for Temperature and Humidity Forecasting

The approach adopted for building predictive models for temperature and humidity is based on a pipeline consisting of an initial transformation of the time series followed by autoregressive modeling. Raw data collected from sensors is first processed to reduce variability and standardize the temporal frequency: the granularity is reduced by computing the average over 30-minute windows, followed by interpolation to fill any gaps. The values are then scaled by a fixed factor (10) and converted into 16-bit integers, a format compatible with the constraints of embedded platforms supported by `emlearn`. This transformation ensures the resulting time series is uniform and ready for training, with a new observation available every 30 minutes, in line with the forecasting horizon required by the system.

Separate predictive models are built for temperature and humidity using a supervised learning approach, where each training example is generated from a window of the previous 24 observations of the same variable. This strategy, known as sliding window, converts the time series into an input-output matrix suitable for regression. The chosen algorithm is the `DecisionTreeRegressor`, which can efficiently learn nonlinear relationships and export the final model in a compact and interpretable format. The decision tree is also ideal for embedded deployment, as it can be directly converted into C code using the `emlearn` library, producing a minimal `.h` header file that contains the model logic optimized for execution on microcontrollers. The resulting system is capable of generating a forecast every 30 minutes, using the previous 24 measurements as input.

The choice of this 30-minute prediction interval proves especially effective for optimizing energy consumption in real-world scenarios. An *anticipatory adjustment* of the heating system, based on the forecasted perceived temperature, allows the system to act before environmental conditions become critical, avoiding reactive peaks that require higher energy. Furthermore, forecasting at regular intervals helps to *stabilize the internal thermal profile*, reducing fluctuations and improving perceived comfort. This is particularly beneficial in systems with *low thermal inertia*, such as heat pumps, which greatly benefit from stable and predictive control.

4 Grafana



Figure 2: Data visualization in Grafana

- **Green line:** actual measurements acquired from physical sensors (temperature and humidity);
- **Yellow line:** predicted values computed locally on the sensor node based on previously collected data.

The top graph corresponds to humidity, while the bottom graph refers to temperature. This dual representation allows for a direct comparison between the real sensor data and the predicted output generated by the machine learning model. This visualization allows an easier evaluation of the accuracy of the algorithm and data visualization.